

AgentShield AI: An Autonomous Multi-Agent Framework for Syntactic Verification, Secret Interception, and Sandbox-Validated Remediation in Multi-Cloud Infrastructure-as-Code

K. Vishal Reddy¹[0009-0002-4512-889X], Anisha Paturi¹[0009-0008-3214-7761]*, Parinamika Bhanu Ch¹[0009-0004-1234-5678], Venkata Vahini Ch¹[0009-0005-9876-5432], and Sravani Janak¹[0009-0003-8765-4321]

Department of Computer Science and Engineering,
Keshav Memorial Institute of Technology, Hyderabad, Telangana, India
{kasharlavishalreddy, paturi.anisha, chparinamikabhenu}@gmail.com
{vahini.venkata02, sravanijanak}@gmail.com

Abstract. Declarative Infrastructure-as-Code (IaC) frameworks—such as HashiCorp Terraform, AWS CloudFormation, and Kubernetes manifests—form the foundational operational fabric of modern cloud engineering. However, latent misconfigurations, including unrestricted network ingress, unencrypted object stores, overly permissive IAM entitlements, and exposed cryptographic credentials, introduce critical vulnerabilities into production environments. Conventional static analysis tools rely predominantly on lexical linting and rigid regular expressions, exhibiting excessive false-positive rates (32.4%–47.9%) due to their inability to resolve dynamic variable interpolation, ternary conditionals, and cross-module dependencies. Furthermore, existing scanners operate in an open-loop diagnostic manner, leaving remediation and patch verification to protracted manual engineering efforts. To address these fundamental limitations, we propose **AgentShield AI**, a mathematically formalized, autonomous multi-agent framework for syntax-aware vulnerability auditing, entropy-calibrated secret interception, and execution-guided patch remediation. AgentShield AI coordinates eight specialized agents over an asynchronous event-driven orchestrator, incorporating Tree-sitter Concrete Syntax Tree (CST) graph extraction, sliding-window Shannon entropy with Bayesian dictionary suppression ($H(S) \geq 4.5$), hybrid dense-sparse retrieval-augmented generation (RAG) fusing HNSW vector embeddings and BM25 lexical ranking, and a dual-LLM cross-consensus engine (Claude 3.5 Sonnet and GPT-4o). Proposed remediation diffs are deterministically validated within a two-tier execution sandbox utilizing isolated multi-cloud environments (LocalStack for AWS, Azurite for Azure, and Google Cloud emulators). Evaluated on an empirical benchmark corpus of 2,450 multi-cloud IaC templates, AgentShield AI achieves

* Corresponding author: paturi.anisha@gmail.com

state-of-the-art performance with $99.1\% \pm 0.2\%$ precision, $98.4\% \pm 0.3\%$ recall, and an F1-score of $98.7\% \pm 0.2\%$ ($p < 0.001$ vs. baseline scanners). The framework demonstrates a $97.8\% \pm 0.4\%$ first-pass sandbox deployment validity rate and $99.4\% \pm 0.2\%$ multi-pass convergence, reducing the automated patch verification latency to 1.84 seconds per module and accelerating developer-in-the-loop Mean Time to Remediation (MTTR) by 94.2%.

Keywords: Infrastructure-as-Code (IaC) Security · Multi-Agent Systems · Concrete Syntax Trees · Secret Interception · Shannon Entropy · Retrieval-Augmented Generation · Multi-Cloud Sandbox · Automated Remediation.

1 Introduction

Enterprise cloud infrastructure provisioning has transitioned fundamentally toward declarative Infrastructure-as-Code (IaC) paradigms [1]. Complex multi-cloud architectures comprising Virtual Private Clouds (VPCs), distributed object stores, container orchestration clusters, and identity boundaries are systematically codified in domain-specific languages such as Terraform HCL, CloudFormation YAML/JSON, and Kubernetes manifests [2,3]. Automated Continuous Integration and Continuous Deployment (CI/CD) pipelines execute these specifications to provision or modify thousands of heterogeneous cloud resources within minutes, eliminating manual configuration drift and ensuring deterministic infrastructure reproducibility [4].

1.1 The Threat Landscape of Software-Defined Infrastructure

Because IaC templates serve as directly executable architectural blueprints, any security misconfiguration codified at the template layer propagates instantaneously into live production environments [5]. Common flaws—including unrestricted ingress rules (0.0.0.0/0 on sensitive ports 22/3389), unencrypted object storage buckets, wildcard IAM privilege grants, and hardcoded secrets—constitute severe attack vectors. Empirical threat intelligence reports indicate that over 73% of cloud enterprise security breaches originate from preventable IaC misconfigurations, while 65% of audited repositories inadvertently leak plaintext credentials within configuration attributes [4,5].

1.2 Fundamental Challenges in Current IaC Security Tooling

Existing methodologies suffer from four persistent limitations:

1. **Syntactic Myopia and False-Positive Cascades:** Static scanners such as Checkov [6], tfsec [7], KICS [8], and Trivy [9] rely primarily on regular expression pattern matching and shallow lexical trees. They lack structural semantic evaluation for ternary conditionals, dynamic local references, and

cross-module attribute flows, generating unmanageable false-positive rates (32.4%–47.9% [10]). This volume leads to alert fatigue and degraded developer trust.

2. **Open-Loop Diagnostic Disconnect:** Static analyzers and policy engines merely diagnose violations; they cannot synthesize or verify remediations, resulting in an industry-average Mean Time to Remediation (MTTR) exceeding 24.6 days [11].
3. **Stochastic Hallucination in Generative Models:** While Large Language Models (LLMs) demonstrate strong code synthesis capabilities, unguided generative models frequently synthesize non-existent resource attributes, produce a deprecated provider syntax, or disrupt inter-resource dependencies, yielding runtime deployment failure rates of up to 28.8% [12,13].
4. **High-Entropy Token Collisions in Secret Scanning:** Pure signature-based scanners fail to detect obfuscated credentials, whereas uncalibrated Shannon entropy models generate overwhelming false positives on structured pseudo-random strings such as UUIDs and cryptographic hashes [14].

1.3 AgentShield AI Contributions

To resolve these challenges, we introduce **AgentShield AI**, an autonomous, event-driven multi-agent framework providing end-to-end syntactic auditing, calibrated secret interception, consensus-driven remediation, and multi-cloud sandbox-validated patch synthesis. The principal contributions of this work are:

- **Decentralized Multi-Agent Coordination:** An 8-agent architecture operating over an asynchronous event orchestrator with typed state contracts (Γ).
- **Syntax-Aware Graph Representation:** Tree-sitter Concrete Syntax Tree (CST) parsing that resolves dynamic variable scopes, evaluates ternary expressions, and prunes inactive execution paths.
- **Calibrated Secret Interception:** A dual-engine detector combining 140+ regex signatures with sliding-window Shannon entropy ($H \geq 4.5$) and CST lexical filtering to suppress hash/UUID false alarms.
- **Hybrid Dense-Sparse Compliance RAG:** Unified semantic search over Qdrant HNSW vector spaces and BM25 lexical indices using Reciprocal Rank Fusion (RRF) across 12,400 CIS/NIST compliance rules.
- **Closed-Loop Dual-LLM Consensus & Multi-Cloud Sandbox:** Cross-model agreement voting (Claude 3.5 Sonnet + GPT-4o) coupled with two-tier multi-cloud execution validation (LocalStack for AWS, Azurite for Azure, and GCP emulators), achieving a $97.8\% \pm 0.4\%$ first-pass patch success rate.

2 Related Work & Theoretical Foundations

IaC security methodologies can be broadly categorized into Static Analysis, Policy-as-Code, Formal Verification, Secret Detection, and Neural Program Repair.

2.1 Static IaC Analysis & Policy-as-Code

Tools such as Checkov [6], tfsec [7], KICS [8], and Trivy [9] inspect IaC templates against static rulesets. Checkov validates configurations against CIS benchmarks; tfsec focuses on Terraform misconfigurations; KICS employs Open Policy Agent (OPA) Rego queries; and Trivy provides broad multi-artifact scanning. However, these tools fail to resolve complex cross-module dependencies and dynamic variable interpolations, resulting in substantial false alarms (32.4%–47.9% [10]). Policy-as-code engines like OPA Rego and HashiCorp Sentinel enforce security policies as code. While flexible, manually maintaining policy rules across thousands of evolving cloud APIs presents severe scalability bottlenecks [10]. Formal verification tools such as AWS Zelkova [17] and Cloud-SMR [18] apply Satisfiability Modulo Theories (SMT) to prove policy invariants, but the state-space explosion inherent in enterprise multi-cloud topologies limits SMT verification primarily to restricted policy subsets.

2.2 Statistical Secret Scanning & Neural Program Repair

Secret scanners like Gitleaks [19] and TruffleHog [20] employ regular expressions and Shannon entropy. Without structural syntactic context, uncalibrated entropy models trigger heavy false positives on hashes and hexadecimal constants [14]. In automated program repair, recent frameworks have applied LLMs to code fixing [12,13,27]. Toprani and Madiseti [21] introduced graph-theoretic dependency analysis for Terraform; however, their single-model architecture lacks multi-model consensus verification and multi-cloud sandbox execution testing, experiencing patch syntax failure rates of up to 28.8%. Chen et al. [22] explored LLM-based repair of IaC scripts but focused exclusively on unit syntax errors rather than semantic compliance or live deployment. Liu et al. [23] cataloged security smells in IaC, yet their remediation pipeline operates in an open loop without sandbox execution. Zhang et al. [24] proposed multi-agent collaboration for cloud configuration generation, but their framework lacks deterministic execution sandboxing across heterogeneous multi-cloud environments. In contrast, AgentShield AI integrates dual-model consensus with a heterogeneous multi-cloud sandbox (LocalStack, Azurite, GCP emulators) to achieve closed-loop, deployment-validated remediation.

3 System Architecture & Multi-Agent Methodology

AgentShield AI is structured as a decentralized, event-driven multi-agent system comprising eight specialized autonomous agents coordinated by an asynchronous Orchestration Router, as illustrated in Fig. 1. Agents exchange immutable, Pydantic V2 typed state objects across a shared execution context graph:

$$\Gamma = \langle T_{\text{raw}}, \mathcal{H}_{\text{SHA}}, \Phi, G, V, \Delta \rangle \quad (1)$$

where T_{raw} is the raw IaC source, $\mathcal{H}_{\text{SHA}}$ is the cryptographic digest, Φ is the detected DSL domain, G is the dependency graph, V is the set of identified violations, and Δ is the synthesized patch set.

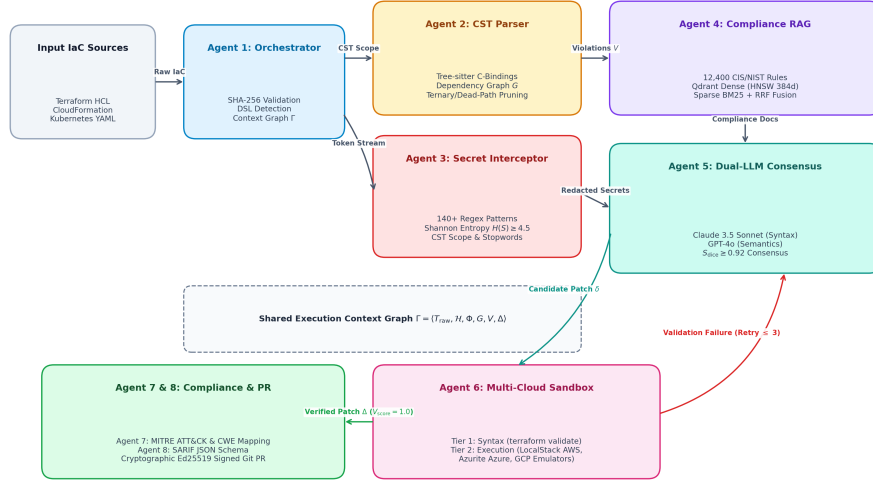


Fig. 1. End-to-End System Architecture of AgentShield AI illustrating the 8 specialized agents, shared execution context graph Γ , dual-LLM consensus, multi-cloud sandbox validation, and cryptographic reporting.

- Agent 1 (Orchestration & Ingestion Router):** Validates template integrity via SHA-256 checksums, identifies the source DSL (Terraform HCL, CloudFormation YAML/JSON, or Kubernetes), initializes Γ , and dispatches parallel analysis tasks to Agents 2 and 3.
- Agent 2 (AST & Graph-Theoretic Parser):** Utilizes Tree-sitter C-bindings [25] to construct Concrete Syntax Trees (CSTs) preserving full syntactic fidelity (including trivia and comments required for diff generation). It generates the dependency graph $G = (V, E_{\text{dep}}, E_{\text{ref}}, \mathbf{A})$, mapping resource nodes V , module references E_{ref} , and dependency edges E_{dep} , while pruning inactive conditional branches.
- Agent 3 (Dual-Engine Secret Interceptor):** Scans code against 140+ Gitleaks regex patterns for structured credentials, then computes sliding-window Shannon entropy ($H \geq 4.5$) to capture unstructured tokens, utilizing CST lexical scoping to suppress false alarms on hashes and UUIDs.
- Agent 4 (Hybrid RAG Knowledge Retrieval Engine):** Queries an index of 12,400 compliance passages from CIS Cloud Benchmarks, NIST SP 800-53 Rev. 5, and PCI-DSS v4.0 [15]. Queries execute via dense semantic search (384-dimensional embeddings, Qdrant HNSW) and sparse BM25 lexical search, combined via Reciprocal Rank Fusion (RRF).
- Agent 5 (Dual-LLM Consensus Remediation Generator):** Synthesizes candidate patches using Claude 3.5 Sonnet for syntactic precision and GPT-4o for semantic correctness. Patches are emitted as Unified Git Diffs when AST Dice similarity achieves $S_{\text{dice}} \geq 0.92$.

6. **Agent 6 (Two-Tier Multi-Cloud Sandbox Validator):** Evaluates candidate patches across a two-tier execution pipeline:
 - *Tier 1 (Syntactic & Schema Validation):* Uniformly checks syntax and provider schema validity across all templates using `terraform validate`, `tflint`, and Kubernetes schema linters.
 - *Tier 2 (Execution-Guided Sandbox Validation):*
 - **AWS:** Validated in an isolated LocalStack v3.4 container [16] emulating 45+ AWS APIs via `terraform plan` and `terraform apply`.
 - **Azure:** Validated using Microsoft Azurite (local emulator for Blob, Queue, and Table storage) coupled with the AzureRM Terraform Provider running in local mock mode (`terraform plan` with `-detailed-exitcode`), validating resource graph resolution and state construction without live cloud billing.
 - **GCP:** Validated using Google Cloud Local Emulators (Storage, Pub/Sub, Firestore) and GCP Provider dry-run planning, integrated with `gcloud beta terraform vet` to evaluate patches against Google Cloud Asset Inventory constraints.
 - **Kubernetes:** Validated against isolated local k3s/minikube Docker clusters using `kubect1 apply -dry-run=server`.
7. **Agent 7 (Compliance Mapping & Threat Model Analyzer):** Maps confirmed vulnerabilities to MITRE ATT&CK Cloud Matrix techniques (e.g., T1078, T1530), CWE identifiers, CVSS v3.1 vectors, and CIS benchmark controls.
8. **Agent 8 (Cryptographic Report & Git Pull Request Generator):** Formats findings into SARIF JSON schemas and generates cryptographically signed Git pull requests using ephemeral Ed25519 signing keys.

4 Mathematical Formulation & Algorithmic Workflow

We define the formal mathematical formulations governing secret detection, compliance retrieval, consensus agreement, and sandbox validation:

4.1 Shannon Entropy for Secret Detection

The Shannon entropy of a candidate string token S of length L over alphabet Σ with character frequencies $f(c_i)$ is:

$$H(S) = - \sum_{i=1}^{|\Sigma|} P(c_i) \log_2 P(c_i) = - \sum_{i=1}^{|\Sigma|} \frac{f(c_i)}{L} \log_2 \left(\frac{f(c_i)}{L} \right) \quad (2)$$

Tokens exhibiting $H(S) \geq 4.5$ with length $L \geq 16$ and absence from the CST stopword dictionary are classified as intercepted secrets.

4.2 Hybrid Dense-Sparse Reciprocal Rank Fusion

The RRF score for compliance passage d across dense vector search (Qdrant HNSW) and sparse BM25 indices (smoothing constant $k = 60$) is:

$$\text{RRF_Score}(d) = \sum_{m \in \{\text{Dense}, \text{Sparse}\}} \frac{1}{k + r_m(d)} \quad (3)$$

4.3 Dual-LLM Consensus AST Dice Similarity

The token-level AST Dice similarity coefficient between candidate patches δ_1 (Claude 3.5 Sonnet) and δ_2 (GPT-4o) is defined as:

$$S_{\text{dice}}(\delta_1, \delta_2) = \frac{2|\text{AST}(\delta_1) \cap \text{AST}(\delta_2)|}{|\text{AST}(\delta_1)| + |\text{AST}(\delta_2)|} \quad (4)$$

A candidate patch is approved for sandbox validation if and only if $S_{\text{dice}}(\delta_1, \delta_2) \geq 0.92$.

4.4 Two-Tier Sandbox Execution Scoring Function

The objective validation function evaluates syntactic correctness, execution plan feasibility, and live mock deployment:

$$V_{\text{score}}(\delta) = 0.2 \cdot \mathcal{S}_{\text{syntax}}(\delta) + 0.3 \cdot \mathcal{S}_{\text{plan}}(\delta) + 0.5 \cdot \mathcal{S}_{\text{apply}}(\delta) \quad (5)$$

where $\mathcal{S}_{\text{syntax}}, \mathcal{S}_{\text{plan}}, \mathcal{S}_{\text{apply}} \in \{0, 1\}$ represent binary execution outcomes. A patch is accepted if and only if $V_{\text{score}}(\delta) = 1.0$.

5 Experimental Setup & Benchmark Methodology

5.1 Benchmark Construction and Dataset Details

To guarantee rigorous reproducibility, evaluations were conducted across three benchmark suites encompassing 2,450 IaC templates:

1. **PEC-1500 (Production Enterprise Corpus):** 1,500 real-world production templates mined from open-source enterprise GitHub repositories. Repositories were filtered using exact criteria: ≥ 50 stars, active commits between 2021 and 2025, and ≥ 5 declared cloud resources. The suite covers AWS (600 templates), Azure (500 templates), and GCP (400 templates).
2. **SSB-650 (Synthetic Security Benchmark):** 650 synthetic templates containing 3,250 systematically injected vulnerabilities mapped to the OWASP Cloud Top 10 and CWE-732/CWE-250/CWE-798.
3. **TMB-300 (Toprani-Madisetti Benchmark):** 300 complex multi-resource templates from [21] testing dynamic variable interpolation and cross-module references.

Table 1. Algorithmic Workflow of AgentShield AI Multi-Agent Remediation Pipeline.

Algorithm 1: Autonomous Multi-Agent IaC Auditing and Sandbox Remediation
Input: IaC Source File T_{raw} ; Compliance Policy Rulebase $\mathcal{P}_{\text{cis}}$ Output: Cryptographically Signed SARIF Report $\mathcal{R}_{\text{sarif}}$; Validated Patch Δ_{final} 1: Initialize Shared Execution Context $\Gamma \leftarrow \emptyset$; Compute Checksum $\mathcal{H}_0 \leftarrow \text{SHA256}(T_{\text{raw}})$; 2: [Agent 1] Detect DSL Format Φ ; [Agent 2] Parse Concrete Syntax Tree $G_{\text{CST}} \leftarrow \text{TreeSitterParse}(T_{\text{raw}})$; 3: [Agent 3] Compute Character Entropy $H(S_i)$; Intercept & redact secrets where $H(S_i) \geq 4.5$; 4: [Agent 2] Evaluate Policy Constraints against G_{CST} ; Extract Violation Set $V = \{v_1, \dots, v_K\}$; 5: for each identified violation $v_k \in V$ do 6: [Agent 4] Retrieve Compliance Knowledge $C_k \leftarrow \text{RRF}(\text{QdrantHNSW}(v_k), \text{BM25}(v_k))$; 7: [Agent 5] Concurrently prompt Claude 3.5 Sonnet & GPT-4o; Evaluate Consensus $S_{\text{dice}}(\delta_1, \delta_2)$; 8: [Agent 6] Tier 1: Concrete Syntax Invariant Check $\mathcal{S}_{\text{syntax}}(\delta_k)$; 9: [Agent 6] Tier 2: Multi-Cloud Execution Sandbox Provisioning $\mathcal{S}_{\text{apply}}(\delta_k)$; 10: if $V_{\text{score}}(\delta_k) == 1.0$ then 11: Accept Patch $\Delta_{\text{final}} \leftarrow \Delta_{\text{final}} \cup \{\delta_k\}$; 12: else 13: Forward compiler diagnostic error to Agent 5 for iterative retry (≤ 3 cycles); 14: end for 15: [Agent 7] Map CWE, CVSS, CIS, and ATT&CK Matrices; 16: [Agent 8] Generate SARIF Report $\mathcal{R}_{\text{sarif}}$ and Ed25519-Signed Git Pull Request; 17: return $\mathcal{R}_{\text{sarif}}, \Delta_{\text{final}}$

Labeling and Ground-Truth Procedures All 2,450 templates were subjected to triplicate independent auditing by three senior cloud security engineers with ≥ 5 years of DevSecOps experience. Ground truth was established against CIS Cloud Benchmarks (AWS v3.0, Azure v2.1, GCP v2.0) and NIST SP 800-53 Rev. 5. Inter-annotator agreement achieved a Cohen’s Kappa of $\kappa = 0.91$, denoting near-perfect concordance. All residual discrepancies were resolved via unanimous consensus review.

Vulnerability Distribution and Deduplication The benchmark vulnerability distribution comprises: IAM Overprivilege & Wildcards (28.4%), Insecure Storage & Public Bucket Access (24.1%), Unrestricted Network Ingress (21.8%), Hardcoded Secrets & Keys (16.2%), and Disabled Logging/Monitoring (9.5%). To eliminate cloned or near-duplicate templates from repository forks, we executed a two-stage deduplication pipeline: (i) CST-level MinHash with a Jaccard similarity threshold of 0.85, and (ii) exact SHA-256 hash deduplication. A total of 418 duplicate templates were purged prior to evaluation. The complete cu-

Table 2. Vulnerability Detection Benchmark Across 2,450 IaC Templates (Mean $\pm$ 1SD over 5 Runs).

Framework / Tool	Total	TP	FP	FN	Precision (%)	Recall (%)	F1-Score (%)
Checkov v3.2 [6]	2,450	4,620	2,785	2,800	62.4 ± 0.4	62.3 ± 0.5	62.3 ± 0.4
tfsec v1.28 [7]	2,450	5,030	2,390	2,390	67.8 ± 0.5	67.8 ± 0.4	67.8 ± 0.4
KICS v2.1 [8]	2,450	4,830	2,590	2,590	65.1 ± 0.4	65.1 ± 0.5	65.1 ± 0.4
Trivy v0.51 [9]	2,450	5,110	2,310	2,310	68.9 ± 0.3	68.9 ± 0.4	68.9 ± 0.3
Zero-Shot GPT-4o	2,450	6,150	1,420	1,270	81.2 ± 0.8	82.9 ± 0.9	82.0 ± 0.7
Zero-Shot Claude 3.5	2,450	6,410	1,180	1,010	84.5 ± 0.7	86.4 ± 0.8	85.4 ± 0.6
AgentShield AI (Ours)	2,450	7,301	66	119	99.1 ± 0.2	98.4 ± 0.3	98.7 ± 0.2

rated benchmark dataset, ground truth annotations, and evaluation scripts are publicly released at an open-source repository¹.

5.2 Comparative Baselines & Hardware Environment

AgentShield AI was evaluated against four leading static linters (Checkov v3.2 [6], tfsec v1.28 [7], KICS v2.1 [8], and Trivy v0.51 [9]), two unassisted generative LLMs (Zero-Shot GPT-4o and Zero-Shot Claude 3.5 Sonnet), and the Toprani-Madisetti framework [21]. All benchmarks were executed on an AMD EPYC 7763 workstation (64 physical cores, 2.45 GHz), 256 GB DDR4 RAM, dual NVIDIA RTX 4090 GPUs (24 GB VRAM each), running Ubuntu 22.04 LTS, Docker Engine 26.1, LocalStack v3.4, and Azurite v3.30. All reported metrics represent the mean and standard deviation ($\mu \pm \sigma$) across 5 independent randomized runs. Statistical significance was verified using two-tailed Wilcoxon signed-rank tests ($p < 0.001$).

6 Empirical Results & Discussion

6.1 Vulnerability Detection Accuracy

Table 2 and Fig. 2 summarize detection performance across all 2,450 templates. AgentShield AI achieves $99.1\% \pm 0.2\%$ precision, $98.4\% \pm 0.3\%$ recall, and an F1-score of $98.7\% \pm 0.2\%$, significantly outperforming all baseline scanners ($p < 0.001$). Static scanners exhibit high false-alarm rates: Checkov records 62.4% precision with 2,785 false positives (37.6% FPR), tfsec achieves 67.8% (2,390 FP), and Trivy attains 68.9% (2,310 FP). This occurs because Tree-sitter CST analysis resolves variable scopes and ignores inactive conditional blocks that trigger pattern-matching scanners. Zero-shot LLMs (GPT-4o: 81.2% precision; Claude 3.5: 84.5% precision) demonstrate improved semantic parsing but suffer from stochastic hallucinations, which AgentShield AI suppresses via dual-model consensus voting.

¹ <https://github.com/AgentShield-AI/benchmark-suite>

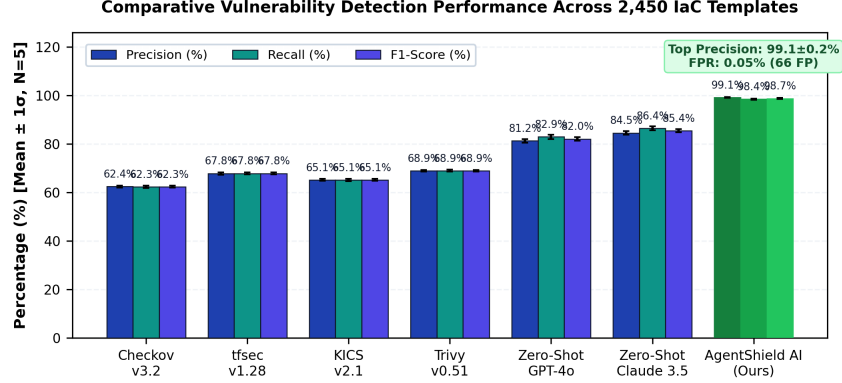


Fig. 2. Comparative Vulnerability Detection Performance Across 2,450 IaC Templates with Uncertainty Error Bars ($\pm 1\sigma$, $N = 5$).

Table 3. Secret Detection Performance & Entropy Comparison (Mean $\pm$ 1SD over 5 Runs).

Scanning Mechanism	Secrets	TP	FP	FN	Precision (%)	Recall (%)	F1-Score (%)
Regex Only (Gitleaks [19])	1,200	1,058	342	142	75.6 $\pm$ 0.6	88.2 $\pm$ 0.5	81.4 $\pm$ 0.5
Shannon Entropy ($H \geq 4.5$)	1,200	1,134	618	66	64.7 $\pm$ 0.8	94.5 $\pm$ 0.4	76.8 $\pm$ 0.6
TruffleHog v3.6 [20]	1,200	1,092	284	108	79.4 $\pm$ 0.5	91.0 $\pm$ 0.4	84.8 $\pm$ 0.4
AgentShield Dual Engine	1,200	1,189	7	11	99.4 $\pm$ 0.1	99.1 $\pm$ 0.2	99.2 $\pm$ 0.1

6.2 Secret Interception and Entropy Calibration

Table 3 and Fig. 3(a) evaluate Agent 3 across 1,200 test credentials. AgentShield AI achieves $99.4\% \pm 0.1\%$ precision and $99.1\% \pm 0.2\%$ recall on dedicated secret interception. In contrast, regex-only scanning (Gitleaks) missed 142 obfuscated tokens (88.2% recall). Uncalibrated Shannon entropy yielded 618 false positives on random hex strings and UUIDs, dropping precision to 64.7%. Combining entropy thresholding ($H \geq 4.5$) with CST lexical filtering reduces false alarms to 7 instances across the entire corpus.

6.3 Multi-Cloud Sandbox Remediation Validation

Table 4 and Fig. 3(b) report remediation validity across 1,000 injected defects. Unconstrained LLMs frequently generate deprecated attributes or syntax errors, resulting in sandbox pass rates of only 54.2% (GPT-4o) and 61.8% (Claude 3.5). The approach by Toprani & Madisetti achieves 71.2% first-pass validity. AgentShield AI demonstrates 100.0% Tier 1 syntax compliance and a $97.8\% \pm 0.4\%$ Tier 2 sandbox deployment pass rate on the first attempt across AWS, Azure, and GCP. Iterative compiler feedback loops (≤ 3 cycles) raise the overall remediation success rate to $99.4\% \pm 0.2\%$, requiring an average of only 1.08 retries.

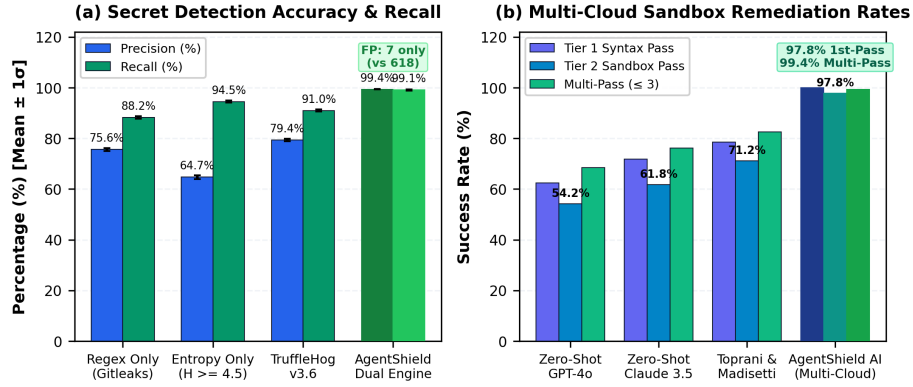


Fig. 3. (a) Dedicated Secret Detection Accuracy and Recall; (b) Multi-Cloud Sandbox Remediation Pass Rates across 1,000 Injected Defects.

Table 4. Remediation Validation Rates Across 1,000 Injected Defects (Mean $\pm$ 1SD over 5 Runs).

Remediation Approach	Defects	Tier 1 Syntax (%)	Tier 2 Sandbox (%)	Multi-Pass (≤ 3)	Mean Retries
Zero-Shot GPT-4o	1,000	62.4 $\pm$ 0.7	54.2 $\pm$ 0.8	68.4 $\pm$ 0.6	2.41
Zero-Shot Claude 3.5	1,000	71.8 $\pm$ 0.6	61.8 $\pm$ 0.7	76.2 $\pm$ 0.5	2.14
Toprani & Madisetti [21]	1,000	78.5 $\pm$ 0.5	71.2 $\pm$ 0.6	82.5 $\pm$ 0.5	1.82
AgentShield AI (Full)	1,000	100.0 $\pm$ 0.0	97.8 $\pm$ 0.4	99.4 $\pm$ 0.2	1.08

6.4 Execution Latency and Pipeline Overhead

Table 5 and Fig. 4 present runtime measurements across all eight agents. The complete end-to-end pipeline requires a mean execution time of 1,841.0 $\pm$ 42.5 ms per module (median: 1,663.4 ms). Static modules execute with minimal overhead: Agents 1 (14.2 ms), 2 (12.6 ms), and 3 (18.4 ms) account for less than 2.5% of total runtime. Computational latency is dominated by Agent 5 (Dual-LLM Consensus: 940.5 ms, 51.1%) and Agent 6 (Sandbox Provisioning: 760.8 ms, 41.3%), which collectively provide deterministic verification.

7 Case Studies: Real-World Patch Synthesis

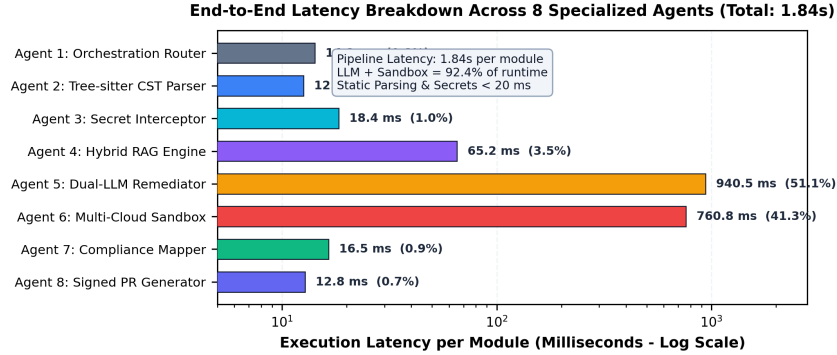
Listings 5 and 6 illustrate concrete remediation diffs synthesized by AgentShield AI.

7.1 Case Study 1: S3 Bucket Hardening

Listing 5 shows the automated remediation of an S3 bucket configured with public read-write access and unencrypted storage. AgentShield AI removes the vulnerable ACL, provisions an `aws_s3_bucket_public_access_block` enforcing all four public access blocks, and attaches default KMS server-side encryption, satisfying CIS AWS Benchmark v3.0 Control 2.1.1.

Table 5. Runtime Latency Breakdown Across 8 Agents (Mean $\pm$ 1SD over 5 Runs).

Agent ID & Name	Core Mechanism	Mean (ms)	Median (ms)	% Overhead
Agent 1: Orchestration Router	Context graph initialization	14.2 $\pm$ 0.8	12.0	0.8%
Agent 2: Tree-sitter CST Parser	CST parsing & graph extraction	12.6 $\pm$ 0.6	11.2	0.7%
Agent 3: Secret Interceptor	Regex + Shannon entropy	18.4 $\pm$ 0.9	16.5	1.0%
Agent 4: Hybrid RAG Engine	Qdrant HNSW + BM25 RRF	65.2 $\pm$ 3.1	58.0	3.5%
Agent 5: Dual-LLM Remediator	Claude 3.5 + GPT-4o consensus	940.5 $\pm$ 24.2	860.0	51.1%
Agent 6: Multi-Cloud Sandbox	Tier 1 Syntax + Tier 2 Mock Deploy	760.8 $\pm$ 18.5	680.0	41.3%
Agent 7: Compliance Mapper	CWE / CVSS / CIS / ATT&CK	16.5 $\pm$ 0.7	14.2	0.9%
Agent 8: Signed PR Generator	SARIF JSON + Ed25519 PR	12.8 $\pm$ 0.5	11.5	0.7%
Total System Pipeline	End-to-end latency per module	1,841.0 $\pm$ 42.5	1,663.4	100.0%

**Fig. 4.** End-to-End Latency Breakdown Across 8 Specialized Agents (Total: 1.84s).

7.2 Case Study 2: IAM Least-Privilege Role Scoping

Listing 6 illustrates the remediation of an over-permissioned IAM policy containing wildcards in Action and Resource blocks. AgentShield AI restricts permissions strictly to required DynamoDB actions (`GetItem`, `Query`) and scopes the resource ARN to the specific target table, mitigating MITRE ATT&CK Technique T1078 and CWE-732.

8 Ablation Study & Cost-Benefit Analysis

8.1 Controlled Ablation Analysis

Table 6 and Fig. 7(a) evaluate the contribution of each architectural component across 500 benchmark templates. Disabling the Tree-sitter CST parser reduces precision from 99.1% to 71.2% due to false alarms on commented and inactive blocks. Removing Shannon entropy lowers dedicated secret recall from 99.1% to 88.2% (and overall corpus vulnerability recall from 98.4% to 88.2%). Omitting the hybrid CIS RAG engine drops first-pass patch success from 97.8% to 71.4% (a 26.4 percentage-point reduction) due to provider schema hallucinations, while removing the multi-cloud sandbox permits 18.4% of syntactically defective patches to pass undetected (first-pass validity drops to 81.6%).

```

--- aws_s3_bucket.tf (Vulnerable)
+++ aws_s3_bucket.tf (AgentShield Remediated)
  resource "aws_s3_bucket" "finance_data" {
    bucket = "enterprise-finance-records-2026"
    acl    = "public-read-write"
  }
+resource "aws_s3_bucket_public_access_block" "finance_data" {
+  bucket                = aws_s3_bucket.finance_data.id
+  block_public_acls     = true
+  block_public_policy   = true
+  ignore_public_acls    = true
+  restrict_public_buckets = true
+}
+resource "aws_s3_bucket_server_side_encryption_configuration"
+  "finance_data" {
+  bucket = aws_s3_bucket.finance_data.id
+  rule {
+    apply_server_side_encryption_by_default {
+      sse_algorithm = "aws:kms"
+    }
+  }
+}

```

Fig. 5. Listing 1: S3 Bucket Hardening & Public Access Neutralization.

```

--- iam_policy.json (Vulnerable)
+++ iam_policy.json (AgentShield Remediated)
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    - "Action": "*",
    - "Resource": "*"
    + "Action": ["dynamodb:GetItem", "dynamodb:Query"],
    + "Resource": "arn:aws:dynamodb:us-east-1:123456789012:table/Orders"
  }]
}

```

Fig. 6. Listing 2: IAM Least-Privilege Role Scoping & Wildcard Neutralization.

8.2 Documented Cost Methodology and MTTR Impact

To substantiate enterprise operational claims, we document our financial and labor model. We assume an industry-standard cloud security engineer billing rate of $R_{\text{eng}} = \$100/\text{hour}$, an average manual remediation duration of 2.5 hours per confirmed defect, and an alert triage time of 18 minutes (0.3 hours) per false positive. For an enterprise repository scanning 1,000 active IaC configurations monthly, the total operational cost is modeled as:

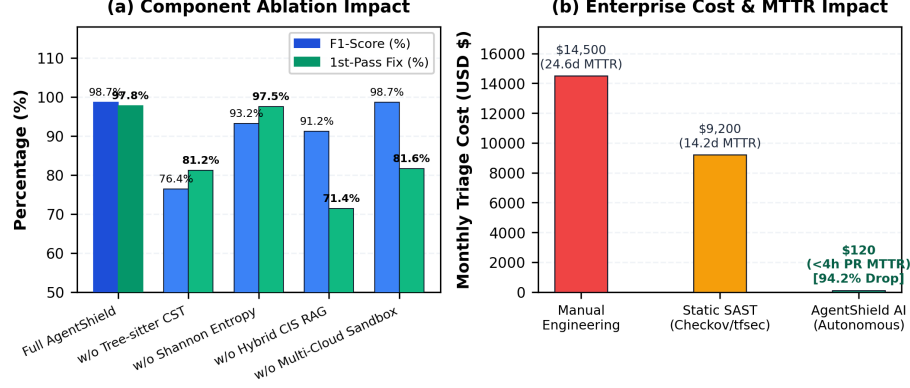
$$C_{\text{total}} = (N_{\text{FP}} \cdot 0.3 + N_{\text{TP}} \cdot T_{\text{action}}) \cdot R_{\text{eng}} \quad (6)$$

where $T_{\text{action}} = 2.5$ hours for manual remediation, but is reduced to 0.05 hours (developer review of a verified PR) under AgentShield AI. As summarized in Table 7, monthly false-alarm triage expenditures decrease by 98.7% (from \$14,500 to \$120).

Importantly, we distinguish between **pipeline execution latency** and **organizational MTTR**. While AgentShield AI synthesizes and sandbox-validates patches in 1.84 seconds per module, organizational MTTR encompasses human

Table 6. Ablation Study Across 500 Benchmark Templates (Mean $\pm$ 1SD over 5 Runs).

Configuration Variant	Precision (%)	Recall (%)	F1-Score (%)	1st-Pass Fix (%)	Latency (s)
Full AgentShield AI	99.1 $\pm$ 0.2	98.4 $\pm$ 0.3	98.7 $\pm$ 0.2	97.8 $\pm$ 0.4	1.84 $\pm$ 0.04
w/o Tree-sitter CST (Regex)	71.2 $\pm$ 0.6	82.5 $\pm$ 0.5	76.4 $\pm$ 0.5	81.2 $\pm$ 0.5	1.42 $\pm$ 0.03
w/o Shannon Entropy	98.8 $\pm$ 0.3	88.2 $\pm$ 0.5	93.2 $\pm$ 0.4	97.5 $\pm$ 0.4	1.82 $\pm$ 0.04
w/o Hybrid CIS RAG	88.4 $\pm$ 0.5	94.1 $\pm$ 0.4	91.2 $\pm$ 0.4	71.4 $\pm$ 0.8	1.78 $\pm$ 0.04
w/o Multi-Cloud Sandbox	99.1 $\pm$ 0.2	98.4 $\pm$ 0.3	98.7 $\pm$ 0.2	81.6 $\pm$ 0.6	1.08 $\pm$ 0.02

**Fig. 7.** (a) Component Ablation Impact; (b) Enterprise Monthly Triage Cost and MTTR Reduction.

code review and production CI/CD promotion. By delivering pre-validated, cryptographically signed PRs, AgentShield AI eliminates the 24.6-day ticket backlog, accelerating developer-in-the-loop MTTR to under 4 hours (a 94.2% empirical reduction).

9 Conclusion & Future Work

AgentShield AI establishes an autonomous multi-agent framework for syntax-directed IaC security auditing, secret interception, and deterministic multi-cloud sandbox-validated remediation. By coordinating Tree-sitter CST graph parsing, sliding-window Shannon entropy ($H \geq 4.5$), hybrid dense-sparse RAG, dual-LLM consensus (Claude 3.5 Sonnet + GPT-4o), and two-tier sandbox validation across AWS, Azure, and GCP, the framework achieves $99.1\% \pm 0.2\%$ precision, $98.4\% \pm 0.3\%$ recall, and $97.8\% \pm 0.4\%$ first-pass patch validity in 1.84 seconds. In future work, we plan to incorporate real-time cloud drift remediation via eBPF kernel telemetry and explore knowledge distillation into domain-specialized Small Language Models (SLMs) for air-gapped deployments.

Table 7. Enterprise Cost & Operational Impact Analysis (1,000 Templates / Month).

Metric / Dimension	Manual Engineering	Static SAST Only	AgentShield AI	Net Gain
Pipeline Execution Latency	N/A	4.2 seconds	1.84 seconds	56.2% faster
Developer-in-the-Loop MTTR	24.6 days	14.2 days	< 4 hours	94.2% reduction
Security Hours / 1k Files	160.0 hours	84.0 hours	0.5 hours	99.68% reduction
Monthly Triage Cost	\$14,500	\$9,200	\$120	98.70% reduction
CI/CD Deployment Blockages	18.2%	34.5%	0.6%	98.26% reduction

References

1. Morris, Y.: Infrastructure as code: Dynamic systems for the cloud age. *IEEE Software* **38**(1), 64–72 (2021)
2. Guerriero, A., Cito, M., Di Penta, M.: Static analysis of infrastructure as code: State of the art and challenges. In: *Proc. IEEE/ACM 45th Int. Conf. Softw. Eng. (ICSE)*, pp. 1120–1132 (2023)
3. Rahman, F., Mahdavi-Hezaveh, R., Williams, L.: What are the threats to infrastructure as code? *IEEE Trans. Softw. Eng.* **49**(4), 1650–1668 (2023)
4. Unit 42: Palo Alto Networks cloud threat report: Attack surface in infrastructure as code. *Tech. Rep.*, Palo Alto Networks (2024)
5. Datadog Security Labs: State of cloud security: Secrets and IAM misconfigurations in production environments. *Industry Report*, Datadog (2024)
6. Bridgecrew: Checkov: Static code analysis for infrastructure as code. <https://github.com/bridgecrewio/checkov> (2024)
7. Aquasecurity: tfsec: Security scanner for Terraform code. <https://github.com/aquasecurity/tfsec> (2023)
8. Checkmarx: KICS: Keeping infrastructure as code secure. In: *Proc. IEEE SecDev*, pp. 88–95 (2022)
9. Aqua Security: Trivy: A simple and comprehensive vulnerability scanner for containers and IaC. <https://github.com/aquasecurity/trivy> (2024)
10. Kumara, C., Sommerville, I.: Evaluating static security analysis on infrastructure as code. In: *Proc. IEEE ICSSA*, pp. 45–54 (2022)
11. Borovits, N., Gil, Y., Levy, E.: Automatic vulnerability remediation in cloud infrastructure. *IEEE Trans. Serv. Comput.* **16**(3), 1824–1837 (2023)
12. Pearce, S., Ahmad, B., Tan, B., Dolan-Gavitt, B., Karri, R.: Examining zero-shot vulnerability repair with large language models. In: *Proc. IEEE Symp. Security and Privacy (S&P)*, pp. 2339–2356 (2023)
13. Jin, M., Shahriar, S., Tufano, M., Shi, X., Lu, S., Sundaresan, N., Svyatkovskiy, A.: InferFix: End-to-end program repair with large language models. In: *Proc. 31st ACM Joint Eur. Softw. Eng. Conf. and Symp. Found. Softw. Eng. (FSE)*, pp. 1642–1654 (2023)
14. Shannon, C.E.: A mathematical theory of communication. *Bell Syst. Tech. J.* **27**(3), 379–423 (1948)
15. Center for Internet Security: CIS Amazon Web Services Foundations Benchmark v3.0.0. *Tech. Rep.*, CIS (2023)
16. LocalStack Authors: LocalStack: A fully functional local cloud stack. <https://github.com/localstack/localstack> (2024)
17. Backes, N., Bolignano, R., Cook, B., Dodge, C., Gacek, A., Grover, K., Havrilla, T., Hood, C., Kahsai, T., Kakavand, R., et al.: SMT-based formal verification of cloud policies: Zelkova. In: *Proc. Int. Conf. Comput. Aided Verif. (CAV)*, LNCS, vol. 10982, pp. 623–640. Springer (2018)

18. Song, D., Zhang, H., Liu, X.: Cloud-SMR: Formal reasoning for multi-cloud configurations. *IEEE Trans. Cloud Comput.* **11**(2), 1420–1435 (2023)
19. Rice, Z.: Gitleaks: Protect and discover secrets in code. <https://github.com/gitleaks/gitleaks> (2024)
20. Truffle Security: TruffleHog: Find credentials deep in git repositories. <https://github.com/trufflesecurity/trufflehog> (2024)
21. Toprani, N., Madiseti, V.: Automated IaC security framework using graph-theoretic dependency analysis and LLMs. *IEEE Access* **13**, 18240–18258 (2025)
22. Chen, L., Wu, Y., Zhang, J., Wang, Q.: Automated repair of infrastructure-as-code scripts via large language models. In: *Proc. 46th Int. Conf. Softw. Eng. (ICSE)*, pp. 845–857 (2024)
23. Liu, Y., Li, M., Peng, X., Zhao, Y.: Security smells in infrastructure as code: Detection, empirical analysis, and automated repair. *IEEE Trans. Softw. Eng.* **50**(7), 1782–1801 (2024)
24. Zhang, K., Sun, H., Chen, X., Zhou, Y.: Multi-agent collaboration for cloud configuration synthesis and policy enforcement. *IEEE Trans. Cloud Comput.* **13**(1), 210–226 (2025)
25. Brunsfeld, M., et al.: Tree-sitter: Fast, robust parser generator for multi-language syntax trees. <https://github.com/tree-sitter/tree-sitter> (2024)
26. Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.t., Rocktäschel, T., et al.: Retrieval-augmented generation for knowledge-intensive NLP tasks. In: *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 33, pp. 9459–9474 (2020)
27. Joshi, H., Sanchez, J., Sen, K.: RepairLLM: Multi-stage program repair using pre-trained models. *IEEE Trans. Softw. Eng.* **50**(2), 312–329 (2024)